Knowledge-Guided Requirement Structure Modeling via Multi-Agent Collaboration

Anonymous Author(s)

Abstract

Natural-language requirements are often loosely structured, semantically mixed, and difficult to analyze in a uniform representation. This paper studies the problem of requirement structure modeling: how to transform informal requirement statements into structured, analyzable, and extensible requirement representations. We outline a knowledge-guided multi-agent workflow that decomposes requirement semantics, constructs a requirement graph, and incorporates human feedback for representation refinement. The paper is positioned as a research-problem-driven study for intelligent requirements engineering, with evaluation centered on structure completeness, semantic consistency, graph quality, and representation accuracy.

Keywords

requirements engineering, requirement structure modeling, requirement graph, structured requirement representation, multi-agent collaboration, knowledge-guided modeling

1 Introduction

1.1 Background and Motivation

Requirements are the primary interface between stakeholder intent and downstream software engineering activities. Recent work has emphasized the central role of requirements in AI-driven software engineering [2]. However, natural-language requirements remain difficult to analyze because their structure is implicit, their semantics are mixed, and their domain assumptions are often left unstated.

1.2 Challenges of Requirement Structure Modeling

This paper focuses on requirement structure modeling rather than on later development activities. We identify four core challenges:

(1) Natural-language requirements mix functions, constraints, roles, data elements, and exception conditions in the same text.
(2) Requirement semantics are ambiguous when stakeholder vocabulary, system behavior, and operational conditions are not explicitly separated.
(3) Existing LLM-based requirement analysis often produces fluent text but lacks a stable structured representation.
(4) Requirement representations must remain inspectable and refinable by humans to support practical requirements engineering.

1.3 Multi-Agent Requirement Parsing

We study a lightweight multi-agent parsing workflow in which different agents focus on elicitation, semantic analysis, and structure construction. The goal is not to build a large agent platform, but to examine whether role-separated collaboration can improve the transformation from informal requirements to a structured requirement graph.

1.4 Contributions

The intended contributions of this paper are:

(1) A research formulation of knowledge-guided requirement structure modeling for natural-language requirements.
(2) A multi-agent workflow for requirement semantic decomposition and structured representation construction.
(3) A requirement graph representation that organizes functional requirements, constraints, roles, data elements, and exception information.
(4) An evaluation design focused on structure completeness, semantic consistency, graph quality, and representation accuracy.

2 Related Work

2.1 Traditional Requirement Modeling

Traditional requirements engineering has developed modeling notations, templates, and controlled languages to reduce ambiguity and improve stakeholder communication. Controlled Natural Language approaches show that structured expression can improve analyzability while preserving readability [1]. Our work follows this motivation but focuses on automatically constructing structured representations from informal natural-language requirements.

2.2 Requirement Representation and Requirement Graphs

Requirement representation research studies how requirement elements and their relations can be made explicit. Graph-based representations are useful because they expose entities, dependencies, conditions, and semantic links in a form that can be inspected and analyzed. This paper treats the requirement graph as the central artifact of requirement structure modeling.

2.3 LLM-based Requirement Analysis

LLM-based methods have been used to summarize, extract, and organize requirement-related artifacts. These methods demonstrate the value of making requirements explicit before using them in later engineering stages. However, fluent generated text does not by itself provide a stable representation for analysis, comparison, or human review.

2.4 Knowledge-Guided Requirement Modeling

Knowledge-guided modeling uses domain vocabulary, requirement templates, semantic patterns, and modeling constraints to guide requirement interpretation. Such knowledge can reduce uncontrolled variation in extracted structures and help align agent outputs with requirements engineering concepts. This paper uses knowledge as

guidance for representation construction rather than as a separate reasoning system.

2.5 Multi-Agent Collaboration

Multi-agent collaboration decomposes complex analysis work into roles with different responsibilities. For requirement structure modeling, role separation can help distinguish stakeholder-oriented elicitation, semantic decomposition, and graph construction. This paper investigates a focused three-role workflow instead of a broad multi-agent software engineering platform.

3 Methodology

3.1 Framework Overview

Given a set of natural-language requirements $R = \{r_1, \ldots, r_n\}$, the goal is to construct a structured requirement representation $G = (V, E, A)$. Nodes V represent requirement elements, edges E represent semantic relations, and attributes A store textual evidence, type labels, confidence, and human revision status. The framework contains three stages: multi-agent parsing, knowledge-guided structure modeling, and human-in-the-loop refinement.

3.2 Multi-Agent Requirement Parsing Workflow

3.2.1 Interviewer Agent. The interviewer agent rewrites stakeholder-facing requirement text into candidate requirement statements and records unresolved assumptions. Its output is a cleaned requirement list rather than a final model.

3.2.2 Analyst Agent. The analyst agent decomposes each candidate requirement into semantic units, including functions, constraints, roles, data elements, and exception information. It also links each extracted unit back to textual evidence.

3.2.3 Structure Modeling Agent. The structure modeling agent maps decomposed semantic units into the requirement graph. It normalizes element types, constructs relations, and prepares the representation for human inspection.

3.3 Requirement Semantic Decomposition

3.3.1 Functional Requirements. Functional requirements describe intended system behaviors, services, or responses to user actions.

3.3.2 Constraints. Constraints capture non-behavioral conditions that restrict how a function should operate, such as quality expectations, policy limits, or operational conditions.

3.3.3 Roles. Roles identify stakeholders, users, external systems, or organizational actors involved in a requirement.

3.3.4 Data Elements. Data elements represent objects, records, attributes, and information items manipulated or referenced by a requirement.

3.3.5 Exception Information. Exception information captures alternative conditions, error cases, missing inputs, or boundary situations stated in the requirement text.

3.4 Knowledge-Guided Structure Modeling

3.4.1 Requirement Knowledge. Requirement knowledge includes domain vocabulary, requirement categories, expected element types, and common relation types.

3.4.2 Structure Templates. Structure templates define the expected schema for decomposed requirement elements and graph relations.

3.4.3 Semantic Patterns. Semantic patterns guide the interpretation of common requirement forms, such as actor-action-object statements and condition-trigger-response statements.

3.4.4 Prompt Strategies. Prompt strategies constrain agent outputs to typed fields, evidence links, and graph construction rules.

3.5 Requirement Graph Construction

3.5.1 Entity Extraction. Entity extraction identifies requirement elements and assigns each element a type, source requirement, and evidence span.

3.5.2 Relation Construction. Relation construction connects entities through typed links such as performs, uses, constrains, triggers, and handles.

3.5.3 Structured Representation. The final representation stores the requirement graph in a structured format that can be inspected, compared, and extended in later requirements engineering activities.

3.6 Human-in-the-Loop Refinement

Human reviewers inspect uncertain nodes, ambiguous relations, and unsupported decompositions. Their edits are recorded as revisions to the graph rather than as unstructured comments, keeping the representation analyzable after refinement.

4 Experiments

4.1 Research Questions

The evaluation is designed around the following research questions:

(1) RQ1: How completely does the workflow recover requirement elements from natural-language requirements?
(2) RQ2: How consistently does the workflow preserve requirement semantics in the structured representation?
(3) RQ3: How does knowledge guidance affect graph quality and representation accuracy?
(4) RQ4: How does human-in-the-loop refinement change the final requirement graph?

4.2 Experimental Setup

This subsection should report the implementation setting, agent prompts, knowledge sources, annotation process, and graph serialization format used in the study. The setup should be sufficient for reproducing the requirement structure modeling workflow.

4.3 Datasets

The datasets should consist of natural-language requirement documents or requirement statements with sufficient domain context for semantic decomposition. For each dataset, the final paper should

report the source, domain, number of requirements, annotation protocol, and any exclusion criteria.

4.4 Baselines

The evaluation should compare the proposed workflow against focused baselines, such as direct LLM extraction, single-agent parsing, and template-only extraction. Baselines should produce the same target representation so that comparison is made at the structure-modeling level.

4.5 Evaluation Metrics

4.5.1 Structure Completeness. Structure completeness measures whether required element types and relations are present in the generated representation.

4.5.2 Semantic Consistency. Semantic consistency measures whether graph elements preserve the meaning of the original requirement text.

4.5.3 Graph Quality. Graph quality evaluates relation validity, node typing, evidence traceability, and structural coherence.

4.5.4 Requirement Representation Accuracy. Representation accuracy compares generated elements and relations against a human-reviewed reference representation.

4.6 Main Results

This subsection should report empirical results for the research questions without introducing unsupported claims. Tables should separate element-level and relation-level performance.

4.7 Ablation Study

The ablation study should examine the contribution of knowledge guidance, role-separated agents, and human refinement. Each ablation should preserve the same input requirements and evaluation metrics.

4.8 Case Study

The case study should trace a small set of requirements from raw text to semantic decomposition and final requirement graph. It should highlight representation decisions, not downstream engineering tasks.

5 Threats to Validity

5.1 Internal Validity

Internal validity threats may arise from prompt sensitivity, inconsistent human annotations, and variation in agent outputs. The study should mitigate these threats through fixed prompts, documented annotation guidelines, repeated runs where appropriate, and adjudication of uncertain cases.

5.2 External Validity

External validity threats concern whether results generalize across domains, requirement styles, and project scales. The evaluation should report dataset characteristics clearly and avoid claiming generality beyond the studied requirement documents.

5.3 Construct Validity

Construct validity threats concern whether the metrics capture the quality of requirement structure modeling. The study should justify the mapping between metrics and the intended properties of structured requirement representation, especially completeness, consistency, and graph quality.

6 Conclusion

This paper studies knowledge-guided requirement structure modeling for intelligent requirements engineering. The proposed paper structure centers on a focused research problem: transforming loose natural-language requirements into a structured, analyzable, and extensible requirement graph. The methodology combines multi-agent requirement parsing, semantic decomposition, knowledge-guided modeling, graph construction, and human-in-the-loop refinement. The experimental design evaluates whether this workflow improves structured requirement representation in terms of completeness, semantic consistency, graph quality, and representation accuracy.

References

[1] Ikram Darif, Ghizlane El Boussaidi, Segla Kpodjedo, and Cristiano Politowski. 2026. Controlled Natural Language for Requirements Specification: A Systematic Literature Review. *ACM Comput. Surv.* 58, 7 (Jan. 2026), 175:1–175:36. doi:10.1145/3778169

[2] Diana Robinson, Christian Cabrera, Andrew D. Gordon, Neil D. Lawrence, and Lars Mennen. 2025. Requirements Are All You Need: The Final Frontier for End-User Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (May 2025), 141:1–141:22. doi:10.1145/3708524

A Appendix

The appendix can include supplementary materials for reproducibility, such as prompt templates, annotation guidelines, graph schemas, and additional examples of requirement graph construction. It should not introduce new research claims beyond the main paper.